THE MAKING OF THE MACHINE LEARNING ENGINEER

Understanding Vertex AI in Bits and Pieces

A Stepwise, Novice-Friendly Training Course for the
Google Cloud Professional Machine Learning Engineer Certification

Eighteen Chapters · Worked Examples · Exam Tips · Practice Questions · Glossary

How to Use This Course

This course was written for one specific reader: someone who has never touched Vertex AI before, but who needs to walk into the Google Cloud Professional Machine Learning Engineer exam room fully prepared. It assumes no prior Google Cloud experience and very little assumed machine learning background beyond curiosity. Every idea is introduced in small, deliberately ordered pieces — "bits and pieces" — so that each new chapter only ever asks you to add one new idea on top of ideas you have already met.

The eighteen chapters are grouped into six parts that mirror the way a machine learning system is actually built in the real world: you start with foundations, move into data, then model development, then pipelines and MLOps, then operating a live system, and finish with a dedicated exam-preparation part that ties every earlier chapter back to the official exam guide.

Throughout the book you will see four repeating visual cues:

- **Exam Tip boxes (green)** — a fact, distinction, or gotcha that has a track record of appearing on the real exam, phrased the way the exam is likely to phrase it.
- **Watch Out boxes (red)** — a common misconception or a trap that catches beginners, explained so you can recognise it instantly.
- **Try It Yourself boxes (blue)** — a short, concrete action you can take in the Google Cloud console or with the gcloud CLI to turn reading into hands-on memory.
- **Summary tables** — dense, at-a-glance comparisons of services, parameters, or options that are easy to confuse.

Each chapter ends with a short recap and, where useful, a small set of self-check questions. Chapter 18 gathers a full 50-question practice exam with answers and explanations, and the appendix collects a one-page cheat sheet you can review the morning of your exam. We recommend working through the chapters in order the first time through, then using the summary tables and cheat sheet for spaced repetition in the final two weeks before your exam date.

TRY IT YOURSELF — Pace yourself

Most successful candidates spend four to six weeks working through material like this, at a pace of one to two chapters a week, combined with hands-on practice in a real (or free-tier) Google Cloud project. Reading alone is not enough — the exam rewards people who have actually clicked the buttons.

Table of Contents

How to Use This Course 2

Table of Contents 3

Welcome to Vertex AI — and to This Exam 10

1.1 What problem is Vertex AI actually solving? . . . 10

1.2 Who is the exam for, and what does it actually test? . . . 10

1.3 How this course is organised . . . 11

Google Cloud Fundamentals for ML Engineers 12

2.1 Projects: the container for everything . . . 12

2.2 Identity and Access Management (IAM) . . . 12

2.3 APIs, quotas, and regions . . . 12

2.4 The core storage and compute building blocks . . . 13

The Machine Learning Lifecycle, End to End 14

3.1 Why start with the whole picture . . . 14

3.2 The eight stages . . . 14

3.3 Mapping lifecycle stages to Vertex AI services . . . 14

3.4 Prototype vs. production: a mindset shift . . . 15

Data Preparation and Ingestion 18

4.1 Where training data lives . . . 18

4.2 Moving and transforming data at scale . . . 18

4.3 Splitting data correctly . . . 18

4.4 Handling missing and imbalanced data . . . 18

Vertex AI Data Labeling 20

5.1 Why labeling is its own discipline . . . 20

5.2 Supported labeling tasks . . . 20

5.3 Human-in-the-loop and active learning . . . 20

Vertex AI Feature Store 21

6.1 The problem Feature Store solves . . . 21
6.2 Core concepts . . . 21
6.3 Point-in-time correctness . . . 21
6.4 Feature Store 2.0 and Feature Registry . . . 21

Vertex AI Workbench 25

7.1 What Workbench actually is . . . 25
7.2 Choosing the right machine . . . 25
7.3 From notebook to production . . . 25

AutoML on Vertex AI 27

8.1 The core idea: low-code, high-quality models . . . 27
8.2 AutoML across data types . . . 27
8.3 BigQuery ML: the SQL-first alternative . . . 27
8.4 When AutoML is the wrong choice . . . 27

Custom Training on Vertex AI 29

9.1 Two ways to bring your own code . . . 29
9.2 Choosing compute: machine types and accelerators . . . 29
9.3 Reading and writing data during training . . . 29
9.4 Reducing cost: spot / preemptible VMs and checkpointing . . . 29

Hyperparameter Tuning and Vertex AI Vizier 31

10.1 Parameters vs. hyperparameters . . . 31
10.2 Vertex AI's built-in tuning service . . . 31
10.3 Search strategies . . . 31
10.4 Making tuning efficient . . . 31

Vertex AI Pipelines 35

11.1 What a pipeline is, and why it matters . . . 35
11.2 Kubeflow Pipelines SDK vs. TFX . . . 35
11.3 Components and artifacts . . . 35

11.4 Scheduling and triggering pipelines . . . 35

Model Registry, Versioning, and CI/CD for ML **37**

12.1 The Vertex AI Model Registry . . . 37

12.2 Why MLOps needs CI/CD, not just automation . . . 37

12.3 Safe rollout patterns . . . 37

Model Deployment and Serving **39**

13.1 Online prediction: Vertex AI Endpoints . . . 39

13.2 Batch prediction . . . 39

13.3 Choosing machine types and scaling for online serving . . . 39

13.4 Private and multi-region serving . . . 40

Model Monitoring, Explainability, and Responsible AI **43**

14.1 Why a model that worked yesterday can fail today . . . 43

14.2 Skew vs. drift — a critical distinction . . . 43

14.3 Vertex Explainable AI . . . 43

14.4 Responsible AI: fairness and bias . . . 43

Vertex AI Vector Search (Matching Engine) **45**

15.1 Why similarity search is a different problem . . . 45

15.2 Where embeddings come from . . . 45

15.3 Core use cases . . . 45

Security, Cost Management, and Quotas **47**

16.1 Securing an ML workload end to end . . . 47

16.2 Protecting sensitive data . . . 47

16.3 Cost management . . . 47

16.4 Budgets, alerts, and labels . . . 48

Exam Domains Mapped to Vertex AI Services **51**

Domain 1 — Architecting low-code ML solutions . . . 51

Domain 2 — Collaborating within and across teams . . . 51

Domain 3 — Scaling prototypes into ML models . . . 51

Domain 4 — Serving and scaling models 51
Domain 5 — Automating and orchestrating ML pipelines 51
Domain 6 — Monitoring, optimizing, and maintaining ML solutions 51

Practice Exam — 40 Questions with Explanations **53**

Glossary **62**

Appendix — One-Page Cheat Sheet **65**

PART 1

Foundations

Why Vertex AI exists, how the exam is structured, and the Google Cloud building blocks every chapter after this one leans on.

CHAPTER 1

Welcome to Vertex AI — and to This Exam

1.1 What problem is Vertex AI actually solving?

Before Vertex AI existed, building a machine learning system on Google Cloud meant stitching together a dozen separate products by hand: AutoML for a quick model, AI Platform for custom training, a hand-rolled TensorFlow Serving cluster for predictions, Cloud Storage buckets for artifacts, and a spreadsheet (usually) to keep track of which model version was actually deployed. Vertex AI, launched in 2021, is Google's answer to that mess: a single, unified platform that covers the entire machine learning lifecycle — from raw data to a monitored production endpoint — under one API, one SDK, one console, and one consistent permission model.

Think of Vertex AI as a toolbox, not a single tool. Some tools in that toolbox write the model for you (AutoML). Some just give you a powerful, managed place to run your own training code (custom training). Some tools store and reuse features across teams (Feature Store). Some tools chain everything into a repeatable, auditable workflow (Vertex AI Pipelines). The exam is fundamentally testing whether you know which tool to reach for in which situation — not whether you can recite every parameter of every API.

1.2 Who is the exam for, and what does it actually test?

The Google Cloud Professional Machine Learning Engineer certification targets someone who can design, build, and productionize ML models on Google Cloud. Google's own exam guide organizes the exam into six domains. You do not need to memorize the exact wording, but you should recognise the shape of each domain, because every chapter in this course is deliberately mapped back to one or more of them (see the table below, and the full cross-reference in Chapter 17).

Exam Domain	Roughly Means…	Where This Course Covers It
1. Architecting low-code ML solutions	Choosing AutoML / BigQuery ML vs. custom code	Ch. 8, 9
2. Collaborating within and across teams	Notebooks, shared code, model hand-off	Ch. 7, 12
3. Scaling prototypes into ML models	Moving from a notebook to a real training job	Ch. 9, 10
4. Serving and scaling models	Endpoints, batch prediction, latency/cost trade-offs	Ch. 13
5. Automating and orchestrating ML pipelines	Vertex AI Pipelines, CI/CD, MLOps	Ch. 11, 12
6. Monitoring, optimizing, maintaining ML solutions	Model Monitoring, Explainable AI, retraining	Ch. 14

EXAM TIP — Read the real exam guide too

This course teaches the concepts thoroughly, but Google periodically refreshes the official exam guide and sample questions. Before your exam date, search for “Google Cloud Professional Machine Learning Engineer exam guide” and skim the current PDF — domain weightings occasionally shift.

1.3 How this course is organised

We move in the same order a real project moves: get the data right first, then develop a model, then build the plumbing that turns a one-off notebook experiment into a repeatable pipeline, then deploy it, then keep it healthy in production. Part 6 circles back and re-sorts everything by exam domain so your final review is organised the way the exam is organised, not the way a project is organised.

1. **Part 1 — Foundations:** Google Cloud building blocks, IAM, and the ML lifecycle as a whole.
2. **Part 2 — Data:** ingestion, storage, labeling, and the Vertex AI Feature Store.
3. **Part 3 — Model development:** notebooks, AutoML, custom training, hyperparameter tuning.
4. **Part 4 — Pipelines & MLOps:** Vertex AI Pipelines, the Model Registry, CI/CD for ML.
5. **Part 5 — Operating ML systems:** deployment, monitoring, explainability, vector search, security.
6. **Part 6 — Exam preparation:** domain cross-reference, a 50-question practice exam, glossary, cheat sheet.

Chapter Recap

- Vertex AI unifies what used to be a dozen separate GCP ML products under one API and console.
- The exam tests judgement — which tool fits which situation — more than raw memorisation.
- The official exam guide has six domains; this course maps every chapter to at least one of them.

CHAPTER 2

Google Cloud Fundamentals for ML Engineers

2.1 Projects: the container for everything

Every single resource you create on Google Cloud — a Vertex AI model, a storage bucket, a BigQuery dataset — lives inside a **project**. A project is the unit of billing, the unit of quota, and (mostly) the unit of permissions. Projects are identified by a globally unique **project ID** (for example, *my-ml-project-472*) and a human-friendly **project name** that can be duplicated across the world. For the exam, remember that a project ID cannot be changed once created, while a project name can.

Projects sit inside optional **folders**, which sit inside an **organization** node, which is the root of everything a company owns on Google Cloud. This hierarchy exists so that policies — such as "nobody may create a publicly readable storage bucket" — can be set once at the top and inherited everywhere below, through a mechanism called **Organization Policy**.

2.2 Identity and Access Management (IAM)

IAM answers exactly one question: *who can do what to which resource?* It is built from three moving parts. A **principal** is the "who" — a human Google account, a Google group, or (most important for ML pipelines) a **service account**, which is an identity a piece of software uses instead of a person. A **role** is a bundle of permissions, such as "can read objects in Cloud Storage". A **role binding** connects a principal to a role on a specific resource, project, folder, or organization.

- **Basic roles** (Owner, Editor, Viewer) are broad and generally discouraged in production — they grant far more than any single job needs.
- **Predefined roles**, like *roles/aiplatform.user* or *roles/aiplatform.admin*, are scoped to a specific service and follow the principle of least privilege.
- **Custom roles** let you assemble your own exact bundle of permissions when no predefined role fits.

Almost everything Vertex AI does on your behalf — running a training job, calling a pipeline step, writing predictions to a bucket — executes under a **service account**, not under your personal identity. By default Vertex AI uses the project's Compute Engine default service account, but production deployments should create a dedicated service account with only the permissions that specific pipeline or model needs.

> **EXAM TIP — Service accounts are a favourite exam topic**
> Expect at least one question along the lines of: "A Vertex AI custom training job needs to read from a Cloud Storage bucket and write to BigQuery. What is the minimum you should do?" The right shape of answer is always: create (or reuse) a dedicated service account, grant it only the specific roles it needs on those specific resources, and attach it to the training job — never widen a basic role at the project level just to make an error go away.

2.3 APIs, quotas, and regions

Every Google Cloud product is exposed as a set of REST APIs that must be explicitly **enabled** per project before use — for Vertex AI, that is the *aiplatform.googleapis.com* API. Enabling an API is a one-time,

project-level action, distinct from granting IAM permission to use it; you need both.

Quotas cap how much of a resource (GPUs, concurrent training jobs, API requests per minute) a project can consume, both to protect Google's infrastructure and to protect you from a runaway bill. Quota increases are requested through the console and are not instant, so provisioning GPU-heavy training capacity should be planned ahead of a deadline, not the day before.

Almost all Vertex AI resources are **regional**: a Vertex AI Feature Store, a training job, and an endpoint each live in one region (for example *us-central1* or *europe-west4*), and, with a few exceptions, cannot directly read data or serve traffic across regions without you explicitly moving or replicating it. Region choice affects latency to your users, data-residency compliance, and which accelerator types (GPU/TPU) are available, since not every accelerator ships in every region.

2.4 The core storage and compute building blocks

Service	Role in an ML Workflow
Cloud Storage	Object storage for raw data, training artifacts, exported models, and pipeline outputs. Vertex AI reads and writes almost everything through *gs://* URIs.
BigQuery	Serverless data warehouse; the default home for large structured/tabular datasets and the engine behind BigQuery ML.
Cloud Storage FUSE / Filestore	Mounting object or file storage as a filesystem inside a training container when code expects local file paths.
Compute Engine / GKE	The underlying virtual machines and Kubernetes clusters that Vertex AI provisions behind the scenes for training and serving — normally invisible to you, but useful to understand when debugging.
Artifact Registry	Stores the custom Docker container images you build for custom training or custom prediction routines.

Chapter Recap

- Projects are the billing/permission container; organizations and folders sit above them.
- IAM connects principals (often service accounts) to roles on resources; least privilege is the exam's preferred answer pattern.
- APIs must be enabled per project; quotas cap consumption and must sometimes be requested in advance.
- Vertex AI resources are regional — pick a region deliberately for latency, compliance, and hardware availability.

CHAPTER 3

The Machine Learning Lifecycle, End to End

3.1 Why start with the whole picture

It is tempting to dive straight into AutoML or custom training, because that is the “fun” part. But almost every exam scenario question describes a business situation and asks you to identify which *stage* of the lifecycle is broken or missing. If you do not have the whole map in your head, it is very easy to pick a technically correct but contextually wrong answer — for example, recommending hyperparameter tuning when the real problem is that the training data is stale.

3.2 The eight stages

1. **Problem framing.** Translate a business goal (“reduce customer churn”) into an ML task (“predict, for each customer, the probability they cancel within 30 days”) with a measurable success metric.
2. **Data collection and ingestion.** Pull raw data from operational systems into a place ML tooling can reach — typically Cloud Storage or BigQuery.
3. **Data preparation.** Clean, label, join, and engineer features; split into train/validation/test sets.
4. **Model development.** Experiment with AutoML and/or custom code, iterating on architecture and features.
5. **Training at scale.** Move from a notebook prototype to a managed, reproducible training job, tuning hyperparameters along the way.
6. **Evaluation.** Measure the model against held-out data and business-relevant metrics, not just accuracy — and check for bias across subgroups.
7. **Deployment and serving.** Package the model behind an endpoint (online) or a batch job, at the latency and throughput the application needs.
8. **Monitoring and maintenance.** Watch for data drift, skew, and degrading performance, and retrain on a schedule or trigger.

In practice these stages are not a straight line — they are a loop. Monitoring in stage eight routinely kicks off a new round of data collection and retraining. Vertex AI Pipelines (Chapter 11) exists specifically to make that loop something you can automate rather than something you redo by hand every quarter.

3.3 Mapping lifecycle stages to Vertex AI services

Lifecycle Stage	Primary Vertex AI / GCP Tooling
Data collection	Cloud Storage, BigQuery, Pub/Sub, Dataflow
Data preparation	Dataflow, BigQuery SQL, Vertex AI Data Labeling, Feature Store
Model development	Vertex AI Workbench, AutoML, BigQuery ML
Training at scale	Vertex AI Training (custom jobs), Vertex AI Vizier (tuning)
Evaluation	Vertex AI Model Evaluation, Explainable AI
Deployment	Vertex AI Endpoints (online), Vertex AI Batch Prediction
Monitoring	Vertex AI Model Monitoring
Orchestration across all stages	Vertex AI Pipelines, Model Registry

3.4 Prototype vs. production: a mindset shift

A recurring exam theme is recognising when a notebook prototype needs to "graduate" into a production system. A prototype optimises for speed of iteration: a data scientist runs cells by hand, hardcodes a file path, and eyeballs a metric. Production optimises for reliability, reproducibility, and cost control: the same training run should produce (nearly) the same result if re-executed, every artifact should be versioned, every step should be able to run unattended, and failures should page someone rather than silently corrupt an endpoint. Vertex AI's tooling exists precisely to bridge that gap — you prototype in Workbench, then encode the finished recipe as a Vertex AI Pipeline.

> **EXAM TIP — A classic exam pattern**
> Watch for questions describing a team whose data scientists "work well in notebooks but struggle to get models into production reliably." The intended answer is almost always to introduce Vertex AI Pipelines (with the Model Registry and CI/CD) so that the notebook's logic becomes a versioned, repeatable, automatically triggered pipeline — not simply "buy a bigger machine."

Chapter Recap

- The ML lifecycle has eight recurring stages, framed as a loop, not a line.
- Every Vertex AI service exists to serve one or more specific stages — memorise the mapping, not just the service names.
- The exam frequently tests whether you can recognise a team stuck at the prototype stage and recommend pipeline-based productionisation.

PART 2

Data

No model is better than the data behind it. This part covers ingestion, labeling, and the Vertex AI Feature Store.

CHAPTER 4

Data Preparation and Ingestion

4.1 Where training data lives

Vertex AI does not force you into one storage format. In practice, three destinations cover almost every real project. **Cloud Storage** holds unstructured data — images, audio, video, free text, and CSV/JSONL/TFRecord files — addressed by *gs://bucket/path* URIs. **BigQuery** holds structured, tabular data at any scale, queried with SQL, and is the natural home for data that already lives in a data warehouse. **Vertex AI managed datasets** are a thin metadata layer on top of either of the above: when you create a managed dataset, Vertex AI tracks its schema, its automatic train/validation/test split, and its lineage back to the models trained from it, without moving or duplicating the underlying data.

4.2 Moving and transforming data at scale

Raw operational data rarely arrives ML-ready. **Dataflow**, Google Cloud's managed Apache Beam service, is the standard tool for large-scale, parallel ETL/ELT: joining tables, deduplicating records, computing aggregates, and writing the result to BigQuery or Cloud Storage. Dataflow pipelines can run in both batch mode (process a fixed dataset once) and streaming mode (process an unbounded stream from Pub/Sub in near real time) — the same programming model covers both, which the exam likes to test by asking you to pick Dataflow over a hand-rolled Compute Engine script for anything that needs to scale.

For data that is already inside BigQuery, plain SQL is often the fastest way to clean and reshape it, especially combined with **BigQuery ML**, which lets you train certain model types directly with SQL statements (covered further in Chapter 8). Pub/Sub sits upstream of both, acting as the durable, scalable message queue that decouples data producers (an app, a device, another service) from data consumers (a Dataflow pipeline, a Vertex AI Feature Store ingestion job).

4.3 Splitting data correctly

Every supervised model needs at least a training set and an evaluation set that the model never sees during training; most projects also carve out a separate test set for a final, one-time check. Vertex AI managed datasets can perform this split automatically (a common default is 80% train / 10% validation / 10% test), or you can specify your own split — for example, a manually assigned column, or a chronological split where the most recent data becomes the test set.

> **WATCH OUT — Time matters more than randomness**
> If your data has a time dimension (transactions, sensor readings, user sessions), a naive random split will leak future information into training and make your evaluation numbers look better than they will be in production. The exam rewards recognising this and recommending a chronological split instead of a random one whenever the scenario mentions time-ordered data.

4.4 Handling missing and imbalanced data

Two data problems show up constantly in exam scenarios. **Missing values** can be imputed (mean/median for numeric columns, a dedicated “unknown” category for categorical columns), dropped if

rare enough, or, for AutoML Tables specifically, left as-is, since AutoML has built-in handling for missing values. **Class imbalance** (for example 99% “not fraud”, 1% “fraud”) can be addressed through resampling (oversampling the minority class or undersampling the majority class), class weighting during training so misclassifying the rare class costs more, or by choosing an evaluation metric that is not misleadingly high on imbalanced data — precision, recall, F1, or the area under the precision-recall curve, rather than raw accuracy.

Chapter Recap

- Cloud Storage holds unstructured data; BigQuery holds structured data; Vertex AI managed datasets add a lineage and split layer on top of either.
- Dataflow (Apache Beam) is the go-to tool for large-scale batch or streaming ETL; Pub/Sub decouples producers from consumers.
- Chronological splits beat random splits whenever the data has a meaningful time dimension.
- For imbalanced classes, prefer class weighting or resampling plus precision/recall-style metrics over raw accuracy.

CHAPTER 5

Vertex AI Data Labeling

5.1 Why labeling is its own discipline

Supervised learning needs labeled examples, and for many real problems — is this X-ray showing a fracture, is this product review positive or negative, where exactly is the pedestrian in this image — no one already has those labels sitting in a database. **Vertex AI Data Labeling** lets you submit unlabeled data together with a set of instructions, and have either your own private workforce or a Google-managed pool of human labelers apply labels at scale.

5.2 Supported labeling tasks

- **Image** — classification (single or multi-label), bounding-box object detection, and segmentation.
- **Video** — classification, object tracking, and action recognition across frames.
- **Text** — classification, entity extraction, and sentiment analysis.

Each task type requires a **labeling instructions document** that you author — essentially a short spec sheet human labelers follow so that two different people label the same example the same way. Poorly written instructions are the single most common cause of low-quality labeled data, and the exam sometimes frames this as “model performance is poor despite a large dataset — what should you check first?” with the answer being label quality and consistency, not model architecture.

5.3 Human-in-the-loop and active learning

For very large unlabeled datasets, labeling everything is often wasteful: many examples are “easy” and a model would learn little from them. **Active learning** patterns — train a small model, use it to flag the examples it is least confident about, send only those to human labelers, retrain — reduce total labeling cost while improving the model faster than labeling at random. While Vertex AI does not fully automate this loop for you out of the box, understanding the pattern is useful both for the exam and for designing a cost-efficient real pipeline.

> **EXAM TIP — Continuous evaluation**
> Vertex AI also supports continuous evaluation, where a small, held-out, human-labeled sample is periodically compared against your deployed model's live predictions. This is one of the tools available for detecting model-quality drift in production, distinct from data-drift monitoring covered in Chapter 14.

Chapter Recap

- Vertex AI Data Labeling supports image, video, and text tasks, using either your own labelers or a Google-managed workforce.
- Clear labeling instructions are the biggest lever on label quality — and label quality is often the real root cause behind a poorly performing model.
- Active learning reduces labeling cost by focusing human effort on the examples a model is least sure about.

CHAPTER 6

Vertex AI Feature Store

6.1 The problem Feature Store solves

In most organisations, the same engineered feature — “average order value over the last 30 days”, say — ends up computed three separate times, slightly differently, by three separate teams: once for a training pipeline, once for a real-time serving system, and once for a batch scoring job. This causes **training-serving skew**: the model was trained on one definition of a feature but sees a subtly different value in production, quietly hurting accuracy in a way that is painful to debug. **Vertex AI Feature Store** is a centralized repository for engineered features that lets every consumer — batch training, online serving, another team entirely — read the exact same feature definition and, importantly, the exact value that was true *at a specific point in time*.

6.2 Core concepts

- **Entity type** — the “thing” features describe, such as *user* or *product*.
- **Feature** — a single named, typed attribute of an entity type, such as *user.average_order_value*.
- **Feature value** — the value of a feature for one specific entity at one specific point in time.
- **Online store** — a low-latency key-value store (backed by Bigtable under the hood) used to fetch the freshest feature values during real-time prediction.
- **Offline store** — a historical record (backed by BigQuery) used for large-scale batch reads during training, including point-in-time lookups.

6.3 Point-in-time correctness

The single most exam-relevant idea in this chapter is **point-in-time correctness**. When you assemble a training set, each label was generated at a specific historical moment, and the features you attach to that label must reflect what was true *at that same moment* — not what is true today. Feature Store's batch serving API accepts a set of entity IDs paired with timestamps and returns, for each pair, the feature values that were correct as of that timestamp, automatically avoiding the classic mistake of accidentally leaking future information into a training set (sometimes called **data leakage** or **feature leakage**).

> **EXAM TIP — “Why did accuracy look great in training but terrible in production?”**
> This exact phrase is a strong signal for either training-serving skew or feature leakage. If the scenario mentions features computed differently for training versus serving, the answer is Feature Store to unify the definition. If it mentions suspiciously high training accuracy that does not hold up, suspect leakage — a feature that, without meaning to, encodes information from after the prediction moment.

6.4 Feature Store 2.0 and Feature Registry

Newer generations of Vertex AI Feature Store decouple feature *definitions* (tracked in a Feature Registry) from where the underlying values physically live, allowing you to register and serve features directly out of existing BigQuery tables (feature views with a BigQuery source) without a separate ingestion step, alongside the classic managed online/offline store model. For the exam, the important concept to retain is

unchanged even as the implementation evolves: one authoritative, versioned definition of each feature, served consistently to both training and online prediction.

Chapter Recap

- Feature Store centralises feature definitions to prevent training-serving skew.
- Online store = low latency, current value, backed by Bigtable-like infrastructure; offline store = historical, point-in-time correct, backed by BigQuery.
- Point-in-time lookups are the mechanism that prevents feature leakage when assembling training sets.

PART 3

Model Development

From a blank notebook to a trained model: Workbench, AutoML, custom training, and hyperparameter tuning.

CHAPTER 7

Vertex AI Workbench

7.1 What Workbench actually is

Vertex AI Workbench is Google Cloud's managed Jupyter notebook environment, purpose-built so that the exploratory work a data scientist does day to day is pre-wired into the rest of Vertex AI — authenticated access to BigQuery and Cloud Storage, one-click submission of a notebook cell as a full Vertex AI Training job, and integration with the Feature Store and Model Registry — without you hand-configuring networking, authentication, or GPU drivers.

Two flavours exist. **Instances** are a single-user JupyterLab environment running on a dedicated virtual machine you can resize, attach GPUs to, and stop when idle to save cost. **Managed notebooks** add automated shutdown on idle, easier integration with source control, and simpler lifecycle management, trading a little configurability for lower operational overhead.

7.2 Choosing the right machine

A Workbench instance is really just a Compute Engine VM with JupyterLab pre-installed, so the same machine-type economics apply: pick a CPU-only, memory-optimised machine for data wrangling with pandas; attach a GPU (commonly an NVIDIA T4 for light work, an A100 for heavy deep learning) only when you are actually iterating on a neural network, and remember to stop — not just close the browser tab for — the instance when you are done, since a running VM bills whether or not you are actively using it.

> **WATCH OUT — The most commonly missed cost-saving answer**
> When a scenario describes a notebook environment left running overnight or over a weekend and racking up cost, the exam's preferred answer is enabling **idle shutdown** (available on managed notebooks and configurable on instances), not “downgrade to a smaller machine” — the waste is time, not size.

7.3 From notebook to production

Workbench is deliberately not the place production training happens. The intended workflow is: prototype interactively in a notebook, extract the working code into a script or container, and submit that as a Vertex AI custom training job (Chapter 9) — either manually the first time, or automatically as one step of a Vertex AI Pipeline (Chapter 11) from then on. Vertex AI Workbench includes a “Submit as training job” action that packages a notebook's code and runs it as a managed job directly, which is a useful bridge while a team is transitioning from purely manual workflows toward full pipelines.

Chapter Recap

- Workbench is managed JupyterLab, pre-integrated with BigQuery, Cloud Storage, Feature Store, and training.
- Instances are more configurable; managed notebooks trade some configurability for automated idle shutdown and easier lifecycle management.

- Idle shutdown is the standard answer to “notebook cost got out of control.”
- Notebooks are for prototyping; production training belongs in a custom training job or a pipeline step.

CHAPTER 8

AutoML on Vertex AI

8.1 The core idea: low-code, high-quality models

AutoML lets you train a production-quality model by supplying labeled data and a target column or label — no model architecture, no hyperparameters, no training loop to write. Behind the scenes, Google's infrastructure searches over model architectures, feature transformations, and hyperparameters on your behalf, using techniques such as neural architecture search and transfer learning from models already trained on very large datasets. AutoML is the default recommendation whenever a scenario describes a team with limited ML expertise, tight timelines, and a fairly standard prediction problem.

8.2 AutoML across data types

AutoML Product	Input Data	Typical Use Cases
AutoML Tables (Vertex AI Tabular Workflows / AutoML)	Structured/CSV/BigQuery data	Churn prediction, demand forecasting, fraud scoring, pricing
AutoML Image	Images	Product defect detection, medical image classification, object detection
AutoML Video	Video clips	Action recognition, content moderation, object tracking
AutoML Text / Natural Language	Text documents	Sentiment analysis, document classification, entity extraction
AutoML Translation	Parallel text corpora	Custom domain-specific translation

8.3 BigQuery ML: the SQL-first alternative

BigQuery ML lets you train a range of model types — linear/logistic regression, k-means clustering, matrix factorization, boosted trees (via XGBoost), and even import of custom TensorFlow models — using ordinary SQL statements (*CREATE MODEL … OPTIONS(model_type='logistic_reg') AS SELECT …*), directly against data that already lives in BigQuery. It is the fastest path from “data in a warehouse” to “working model” for teams whose strongest skill is SQL, and BigQuery ML models can be registered into the Vertex AI Model Registry and deployed to a Vertex AI endpoint just like any other model.

EXAM TIP — AutoML Tables vs. BigQuery ML — how the exam distinguishes them
Both handle tabular data, but the exam expects you to know the emphasis: AutoML Tables optimises automatically across many model families for the best possible accuracy with almost no configuration; BigQuery ML optimises for staying entirely inside SQL and BigQuery, trading a little automatic sophistication for tight integration with a data warehouse a SQL-fluent team already knows well.

8.4 When AutoML is the wrong choice

AutoML is not always the right answer, and the exam tests this directly. Prefer custom training instead when: the problem needs a novel or research-grade architecture AutoML does not offer; you need to reuse or fine-tune a very specific pretrained model; regulatory or interpretability requirements demand full visibility into model internals AutoML abstracts away; or the training data is small enough, or the problem specialised enough, that a carefully hand-tuned custom model can outperform a general-purpose automated search.

- AutoML trades control for speed and low required expertise — good default for standard problems.
- Custom training trades speed and required expertise for full control — good default for novel, highly specialised, or research-driven problems.
- A common middle ground is starting with AutoML to get a fast baseline, then moving to custom training only if AutoML's accuracy is not sufficient.

Chapter Recap

- AutoML automates architecture search, feature engineering, and hyperparameter tuning for image, video, text, and tabular data.
- BigQuery ML trains models with SQL directly against warehouse data — fast for SQL-fluent teams.
- Prefer custom training when you need architecture control, interpretability, or a highly specialised model.

CHAPTER 9

Custom Training on Vertex AI

9.1 Two ways to bring your own code

Vertex AI Training runs your training code on managed infrastructure without you provisioning a single VM by hand. There are two on-ramps. A **pre-built container** is a Google-maintained Docker image already containing a specific version of TensorFlow, PyTorch, scikit-learn, or XGBoost; you supply only a Python training script, and Vertex AI handles the rest. A **custom container** is a Docker image you build yourself, useful when you need a specific combination of libraries, a non-Python language, or system dependencies a pre-built image does not include. Both approaches are submitted the same way, as a **CustomJob** (a single training run) or, more commonly for teams, wrapped in a reusable **TrainingPipeline** that also handles the resulting model's upload to the Model Registry.

9.2 Choosing compute: machine types and accelerators

Every custom training job specifies a **worker pool** — the machine type, the count of workers, and, optionally, the type and count of accelerators (GPUs such as NVIDIA T4, V100, A100, or TPUs). For single-machine training, you specify one worker pool. For **distributed training**, you specify multiple worker pool specs — typically a chief/primary pool plus one or more worker pools — and your training code must itself be written with a distribution strategy (for example TensorFlow's *MultiWorkerMirroredStrategy* or PyTorch's *DistributedDataParallel*) that knows how to split work across those machines.

Scenario	Recommended Approach
Small-to-medium tabular/text model, standard framework	Pre-built container, single worker, CPU or one GPU
Large deep learning model, single powerful machine is enough	Pre-built or custom container, one worker pool, multiple GPUs on that machine
Very large deep learning model, needs multiple machines	Custom container with a distribution strategy, multiple worker pools
Exotic dependencies / non-Python stack	Custom container

9.3 Reading and writing data during training

Vertex AI automatically sets environment variables inside a training job — *AIP_MODEL_DIR*, *AIP_CHECKPOINT_DIR*, and *AIP_TENSORBOARD_LOG_DIR* — pointing at Cloud Storage locations your script should write to, so that model artifacts, checkpoints, and training logs survive after the job's compute is torn down. Well-written training scripts read these environment variables rather than hardcoding paths, which is exactly what lets the same script run unmodified whether it is launched from a notebook, a CLI command, or a pipeline step.

9.4 Reducing cost: spot / preemptible VMs and checkpointing

Training jobs can request **Spot VMs** (the modern name for what were previously called preemptible VMs), which cost substantially less than standard on-demand pricing in exchange for Google being able to reclaim the machine with little notice. This is an excellent trade-off for training jobs that are not time-critical and whose code checkpoints progress regularly — if reclaimed, the job resumes from the last checkpoint rather than restarting from scratch.

> **EXAM TIP — "How do I reduce training cost without sacrificing correctness?"**
> When a scenario asks for a cheaper way to run non-urgent training, and does not mention a hard deadline, Spot VMs combined with regular checkpointing is almost always the intended answer — far more often than "use a smaller machine", which risks slowing convergence or running out of memory.

Chapter Recap

- Pre-built containers cover common frameworks with just a script; custom containers give full control.
- Worker pools define compute; multiple worker pools plus a distribution strategy enable distributed training across machines.
- AIP_MODEL_DIR and friends are the standard, portable way to read/write artifacts from inside a job.
- Spot VMs plus checkpointing is the standard answer for reducing non-urgent training cost.

CHAPTER 10

Hyperparameter Tuning and Vertex AI Vizier

10.1 Parameters vs. hyperparameters

A model's **parameters** (weights, biases, split thresholds) are learned automatically from data during training. Its **hyperparameters** — learning rate, number of trees, number of layers, regularisation strength, batch size — are set before training begins and are not learned from data at all; they control *how* learning happens. Choosing good hyperparameters can be the difference between a model that converges quickly to a strong result and one that never converges at all.

10.2 Vertex AI's built-in tuning service

Vertex AI Hyperparameter Tuning wraps your custom training job in an outer search loop: you declare which hyperparameters to search over, their type (integer, double/continuous, categorical, discrete) and their valid range, an objective metric to optimise (such as validation accuracy or loss), and a maximum number of trials. Vertex AI then launches many training-job trials, feeding each one a different hyperparameter combination, and tracks the best result. Under the hood this service is powered by **Vertex AI Vizier**, a black-box optimisation service that also works standalone for tuning things that are not even ML models — for example, tuning a physical experiment's parameters.

10.3 Search strategies

- **Grid search** — exhaustively tries every combination on a fixed grid. Simple and thorough, but the number of trials explodes combinatorially as you add hyperparameters, making it impractical beyond a small number of dimensions.
- **Random search** — samples combinations at random. Often outperforms grid search for the same trial budget, because it explores each hyperparameter's range more efficiently.
- **Bayesian optimisation** — Vertex AI Vizier's default approach: builds a probabilistic model of how hyperparameters relate to the objective metric and uses it to choose the next trial's parameters intelligently, converging on good regions faster than either grid or random search, especially when training is expensive.

10.4 Making tuning efficient

Two features keep tuning from becoming an expensive, slow exercise. **Parallel trials** let several trials run at once instead of strictly one after another, trading a bit of search efficiency (later trials cannot yet benefit from the very latest results) for wall-clock speed. **Early stopping** lets Vertex AI abandon a trial partway through if its intermediate results are clearly worse than other trials, freeing that compute for a more promising trial instead of wasting it on a run headed nowhere.

> **EXAM TIP — Fewer hyperparameters, faster convergence**
> A frequently tested judgement call: if tuning is taking too long or costing too much, the fix is rarely “add more trials”. It is almost always to narrow the search space — tune fewer hyperparameters at once, tighten their ranges based on domain knowledge, enable parallel trials, and enable early stopping.

Chapter Recap

- Hyperparameters are set before training and control how learning happens; parameters are learned from data.
- Vertex AI Hyperparameter Tuning wraps a custom training job in a search loop, powered by Vertex AI Vizier.
- Bayesian optimisation (Vizier's default) generally beats grid and random search for expensive training jobs.
- Parallel trials and early stopping are the standard levers for making tuning faster and cheaper.

PART 4

Pipelines and MLOps

Turning a working notebook recipe into an automated, versioned, auditable production workflow.

CHAPTER 11

Vertex AI Pipelines

11.1 What a pipeline is, and why it matters

A **Vertex AI Pipeline** is a directed acyclic graph (DAG) of containerised steps — for example, “extract data” → “validate schema” → “train model” → “evaluate” → “conditionally deploy if accuracy improved” — that Vertex AI executes as a managed, serverless workflow. Each step runs in its own container, its inputs and outputs are automatically tracked as artifacts, and the entire run is versioned and reproducible: you can point at any historical pipeline run and see exactly what data, code, and parameters produced a given model.

Pipelines solve the “it worked on my laptop” problem at the level of an entire workflow, not just a single script. This is the backbone of MLOps on Vertex AI, and it is the mechanism that turns “retrain the model when new data arrives” from a manual chore into an automated, auditable event.

11.2 Kubeflow Pipelines SDK vs. TFX

Vertex AI Pipelines can execute DAGs authored with either of two open-source SDKs. The **Kubeflow Pipelines (KFP) SDK** is a general-purpose, Python-decorator-based way to define pipeline components and connect them — flexible enough for any workflow, ML or not. **TensorFlow Extended (TFX)** is a more opinionated, ML-specific pipeline framework with pre-built components for common ML steps (ExampleGen for ingestion, StatisticsGen and SchemaGen for data validation, Trainer, Evaluator, Pusher for deployment). Both compile down to the same underlying pipeline specification format that Vertex AI Pipelines executes, so the choice is about authoring convenience, not runtime capability.

	Kubeflow Pipelines SDK	TFX
Best for	Custom or mixed (non-TensorFlow) workflows	Standardised TensorFlow-centric ML workflows
Component style	You write each component's logic yourself	Rich library of pre-built, battle-tested ML components
Learning curve	Lower to start, more to build by hand	Higher to start, less to build by hand

11.3 Components and artifacts

A **component** is the smallest reusable unit of a pipeline — a containerised piece of code with declared inputs and outputs. Components can be defined as lightweight Python functions (KFP compiles the function into a container automatically), or as fully custom containers when you need specific dependencies. **Artifacts** — datasets, models, metrics, evaluation results — flow between components and are automatically logged to **Vertex ML Metadata**, giving you lineage: for any model, you can trace back exactly which data, code version, and hyperparameters produced it.

11.4 Scheduling and triggering pipelines

Pipelines can be triggered manually, run on a **Cloud Scheduler**-driven cron schedule (for example, retrain every Sunday night), or triggered by an event — commonly, a new file landing in a Cloud Storage bucket, via Eventarc, or a CI/CD system kicking off a run after a code change is merged. This is the mechanism that closes the MLOps loop described in Chapter 3: monitoring detects drift, which triggers a pipeline run, which retrains and, if the new model passes evaluation, redeploys it — with no human in the loop for routine retraining.

> **EXAM TIP — "Automate model retraining" is a Pipelines question in disguise**
> Any scenario asking how to automate retraining, ensure reproducibility, or give a team an auditable history of every model version is pointing you toward Vertex AI Pipelines (usually paired with the Model Registry from Chapter 12) — not toward a bigger notebook or a cron job that just re-runs a loose script.

Chapter Recap

- A Vertex AI Pipeline is a DAG of containerised, versioned, reproducible steps.
- KFP SDK is general-purpose and flexible; TFX is opinionated and ML-specific with rich pre-built components.
- Components pass artifacts to each other; Vertex ML Metadata automatically records full lineage.
- Pipelines can be triggered on a schedule or by an event, closing the monitor → retrain → redeploy loop.

CHAPTER 12

Model Registry, Versioning, and CI/CD for ML

12.1 The Vertex AI Model Registry

The **Model Registry** is the single source of truth for every model your organisation has trained on Vertex AI, regardless of whether it came from AutoML, custom training, or BigQuery ML. Each entry tracks **versions** of a logical model (so “churn-predictor” can have v1, v2, v3…), evaluation metrics attached to each version, and which version, if any, is currently deployed to which endpoint(s). Aliases (for example, tagging one version as *“default”* or *“champion”*) let downstream systems always refer to “the current production model” without hardcoding a specific version number.

12.2 Why MLOps needs CI/CD, not just automation

Automating a pipeline run is necessary but not sufficient for trustworthy MLOps. **Continuous integration** for ML means every change to training code, pipeline definitions, or feature engineering logic is automatically tested — unit tests on transformation code, a smoke-test training run on a tiny sample of data — before it can affect production. **Continuous delivery/deployment** means a newly trained model that passes evaluation thresholds is automatically promoted toward production through a controlled, auditable process, rather than someone manually copying a model file to a server.

On Google Cloud this is typically implemented with **Cloud Build** (or another CI system) triggered by a commit to a source repository, running tests, building and pushing any custom training/serving containers to **Artifact Registry**, and then triggering a Vertex AI Pipeline run. The pipeline's own final steps handle conditional promotion: register the new model version, compare its evaluation metrics against the currently deployed version, and only redeploy the endpoint if the new version is actually better.

12.3 Safe rollout patterns

Even a model that passed offline evaluation can behave unexpectedly against live traffic, so production MLOps borrows deployment-safety patterns from general software engineering.

- **Canary deployment** — route a small percentage of live traffic to the new model version while the rest continues to the old version, watching for problems before a full rollout.
- **Shadow deployment (a.k.a. shadow / dark launch)** — run the new version alongside the old one on the same traffic, but only ever serve the old version's predictions to users, comparing the new version's predictions offline before it ever affects anyone.
- **Traffic splitting** — Vertex AI Endpoints natively support splitting a percentage of traffic across multiple deployed model versions on the same endpoint, which is the underlying mechanism that makes canary rollouts practical.
- **Rollback** — because every version stays in the Model Registry, reverting a bad rollout is simply re-pointing the endpoint's traffic split back to the previous version.

WATCH OUT — Canary vs. shadow — a common point of confusion

Both reduce risk, but they differ in whether users are exposed. Canary *does* expose a small slice of real users to the new model. Shadow deployment exposes zero users — the new model's predictions are logged and compared, never served. If a scenario says “without impacting any real users”, the answer is shadow deployment, not canary.

Chapter Recap

- The Model Registry tracks every model version, its metrics, and its deployment status in one place.
- CI/CD for ML tests and gates changes to training code and pipelines before they affect production models.
- Canary and shadow deployments both reduce rollout risk; the key difference is real-user exposure.
- Because versions persist in the registry, rollback is just re-splitting endpoint traffic.

CHAPTER 13

Model Deployment and Serving

13.1 Online prediction: Vertex AI Endpoints

An **endpoint** is a persistent, managed HTTPS service that hosts one or more deployed model versions and returns predictions synchronously, typically in well under a second. Deploying a model to an endpoint provisions machine resources (CPU, memory, optionally GPUs) that stay running and billed continuously, which is what makes online prediction fast — there is no cold start for each request. Endpoints support **autoscaling** between a minimum and maximum replica count based on traffic, and, as covered in Chapter 12, can split traffic across multiple deployed versions for safe rollouts.

13.2 Batch prediction

Batch prediction takes a large input dataset (a Cloud Storage file or a BigQuery table), runs every record through the model, and writes results back out, then shuts the underlying compute down. There is no persistent endpoint and no per-request latency guarantee — you get results some time later, at a much lower cost per prediction, because you only pay for the compute while the job actually runs. Batch prediction is the right tool whenever predictions do not need to be immediate: nightly churn scoring for the entire customer base, monthly risk re-scoring, bulk content classification.

	Online Prediction (Endpoint)	Batch Prediction
Latency	Milliseconds, synchronous	Minutes to hours, asynchronous
Cost model	Pay for always-on resources	Pay only while the job runs
Typical trigger	A live user action or API call	A schedule, or a large accumulated dataset
Example	Fraud check on a single transaction as it happens	Score every customer for churn risk once a week

WATCH OUT — Do not default to an endpoint
A very common exam trap describes a batch use case (“score all 10 million customers once a week”) and offers “deploy to an endpoint and loop over every customer” as a tempting but wrong answer. The efficient, intended answer is batch prediction — no idle endpoint cost, and built for exactly this kind of bulk, latency-tolerant workload.

13.3 Choosing machine types and scaling for online serving

Endpoint machine choice follows the same logic as training: CPU-only machines suffice for lightweight models (most tabular AutoML and BigQuery ML models); GPUs help when the model itself is compute-heavy at inference time, such as large deep learning models for images or text. Autoscaling parameters — minimum replica count, maximum replica count, and a target utilisation metric — balance latency against cost: a minimum of zero saves money on rarely used endpoints but introduces cold-start latency on the first request after idle; a minimum of one or more avoids that cold start at the expense of

paying for at least one replica at all times.

13.4 Private and multi-region serving

For workloads with strict latency or compliance requirements, Vertex AI Endpoints support **Private Service Connect** so that prediction traffic never traverses the public internet, staying entirely within a VPC. For global applications, deploying the same model to endpoints in multiple regions, fronted by a global load balancer, keeps latency low for geographically distributed users and adds resilience against a single-region outage.

Chapter Recap

- Endpoints serve online predictions with low latency but continuous cost; batch prediction is cheaper and asynchronous, ideal for bulk or scheduled scoring.
- Autoscaling min/max replicas trade cold-start latency against idle cost.
- GPUs help compute-heavy inference (deep learning); CPU is usually enough for typical tabular models.
- Private Service Connect keeps prediction traffic off the public internet for latency- or compliance-sensitive workloads.

PART 5

Operating ML Systems

Keeping a deployed model healthy, explainable, fast to search, secure, and affordable.

CHAPTER 14

Model Monitoring, Explainability, and Responsible AI

14.1 Why a model that worked yesterday can fail today

A model is a snapshot of patterns learned from historical data. The world keeps moving after that snapshot is taken — customer behaviour shifts, a pandemic changes shopping patterns, a competitor changes the market, a sensor is recalibrated — and the model's assumptions quietly stop holding. **Vertex AI Model Monitoring** continuously compares live prediction traffic against the data the model was trained on (or against a recent baseline) to catch this before it becomes a business problem instead of after.

14.2 Skew vs. drift — a critical distinction

- **Training-serving skew** — the statistical distribution of live serving data diverges from the distribution of the training data. This is measured once, comparing production traffic back to the original training set, and often points to a data pipeline bug or an unrepresentative training set.
- **Prediction drift** — the statistical distribution of live serving data diverges over time from *itself* — this week's traffic compared to last month's. This is measured continuously and typically reflects genuine real-world change rather than a one-off bug.

Both are measured using statistical distance metrics between distributions — commonly, Jensen-Shannon divergence for numerical features and L-infinity distance for categorical features — compared against a threshold you configure. When a monitored feature crosses its threshold, Model Monitoring raises an alert, which in a mature MLOps setup triggers the retraining pipeline from Chapter 11 automatically.

> **EXAM TIP — Skew is measured once against training data; drift is measured continuously against itself**
> A reliable way to answer skew-vs-drift questions on the exam: if the comparison baseline mentioned in the scenario is “the original training data,” it is skew. If the baseline is “recent past production traffic,” it is drift.

14.3 Vertex Explainable AI

Explainable AI answers “why did the model predict this?” for an individual prediction, attributing a share of the outcome to each input feature. For tabular and simpler models, Vertex AI uses **Shapley values** (a game-theoretic method that fairly distributes credit for a prediction among the input features). For image models, **Integrated Gradients** or **XRAI** produce a saliency map highlighting which pixels most influenced the prediction. Explanations serve two audiences: engineers debugging a model that behaves oddly, and end users or regulators who need a defensible reason behind an automated decision, such as a loan denial.

14.4 Responsible AI: fairness and bias

A model can be highly accurate overall while performing much worse for a specific subgroup — for example, a face detection model trained mostly on one demographic performing worse on others. Responsible AI practice on Vertex AI means evaluating performance *sliced* by relevant subgroups, not only in aggregate, and using fairness metrics (such as equality of false-positive rates across groups) alongside raw accuracy. The **Data Card** and **Model Card** patterns — short, standardised documents describing a dataset's or model's intended use, known limitations, and evaluation results — are a practical way to record and communicate this analysis to other teams.

Chapter Recap

- Model Monitoring compares production data against training data (skew) or against itself over time (drift), using statistical distance metrics.
- Explainable AI attributes a prediction to individual input features (Shapley values for tabular, Integrated Gradients / XRAI for images).
- Responsible AI means evaluating and reporting model performance sliced by subgroup, not only in aggregate.

CHAPTER 15

Vertex AI Vector Search (Matching Engine)

15.1 Why similarity search is a different problem

Traditional databases find exact or rule-based matches: “WHERE user_id = 42”. Many modern ML applications instead need to find items that are *semantically similar* — the products most like this one, the support articles most relevant to this question, the images that look most like this image. The standard technique is to convert each item into a dense numeric **embedding vector** (using a trained model), such that similar items end up close together in that vector space, and then search for the nearest vectors to a query vector.

Vertex AI Vector Search (the current name for what was originally launched as Matching Engine) is a managed service for exactly this: given millions or billions of embedding vectors, it finds the approximate nearest neighbours to a query vector in milliseconds, using an efficient approximate nearest-neighbour (ANN) algorithm rather than a brute-force scan of every vector, which would not scale.

15.2 Where embeddings come from

Vector Search does not create embeddings itself — it indexes and searches vectors you supply. Embeddings typically come from a model trained or fine-tuned for that purpose (for example, a text embedding model, an image embedding model, or a two-tower recommendation model), often served through Vertex AI's prediction endpoints or a foundation model's embedding API. The typical pipeline is: run your catalogue of items through an embedding model once (batch), build a Vector Search index from the resulting vectors, then at query time embed the incoming query the same way and ask Vector Search for its nearest neighbours.

15.3 Core use cases

- **Semantic search** — returning documents or passages that match a query's meaning, not just its keywords.
- **Recommendation** — “items similar to this one” or “users similar to this user”, common in retail and media.
- **Retrieval-augmented generation (RAG)** — retrieving the most relevant documents to feed as context into a large language model prompt, grounding its answer in real, current information rather than relying solely on what the model memorised during training.
- **Deduplication and anomaly detection** — finding near-duplicate records, or records that have no close neighbours at all.

WATCH OUT — Vector Search vs. Feature Store — do not mix these up

Both are described as “stores” and both appear in exam scenarios about serving data quickly, but they solve different problems. Feature Store answers “what is the current value of this named feature for this entity?” — an exact key lookup. Vector Search answers “which items are most similar to this one?” — an approximate nearest-neighbour search over embeddings. If the scenario mentions similarity, recommendations, or semantic search, think Vector Search; if it mentions a named feature value for training or serving, think Feature Store.

Chapter Recap

- Vector Search performs fast approximate nearest-neighbour search over embedding vectors at scale.
- It indexes vectors you supply; an embedding model (yours or a foundation model's) produces those vectors.
- Typical uses: semantic search, recommendations, RAG retrieval for LLMs, and deduplication.
- Do not confuse it with Feature Store — similarity search vs. exact feature-value lookup.

CHAPTER 16

Security, Cost Management, and Quotas

16.1 Securing an ML workload end to end

Security for an ML system spans the same surface as any other cloud application, plus a few ML-specific concerns. **Data security**: encrypt data at rest (Google Cloud does this by default, and Customer-Managed Encryption Keys, or CMEK, are available when a project requires control over the encryption key itself) and in transit (TLS by default). **Access security**: apply the least-privilege IAM patterns from Chapter 2 to every service account a training job, pipeline, or endpoint runs under. **Network security**: use VPC Service Controls to create a security perimeter that prevents data exfiltration even by an otherwise-authorised identity, and Private Service Connect / Private Google Access to keep traffic off the public internet.

16.2 Protecting sensitive data

When training data contains personally identifiable information (PII) — names, national ID numbers, health records — the **Sensitive Data Protection** service (formerly known as Cloud Data Loss Prevention, or DLP) can automatically discover and classify that data, and either redact, mask, or tokenise it before it ever reaches a training pipeline. This is frequently the intended answer whenever an exam scenario mentions handling healthcare, financial, or other regulated data.

16.3 Cost management

Vertex AI costs accumulate from several independent sources: training compute (billed per second, more for GPUs/TPUs), online endpoint compute (billed continuously while replicas are running, whether or not they are receiving traffic), batch prediction compute (billed only while the job runs), storage, and AutoML's own per-node-hour training charges. The practical cost levers a well-prepared candidate should recognise are: prefer batch prediction over an always-on endpoint for latency-tolerant workloads (Chapter 13), use Spot VMs with checkpointing for non-urgent training (Chapter 9), set a sensible minimum replica count and enable idle shutdown on notebooks (Chapter 7), and narrow hyperparameter search spaces rather than brute-forcing large ones (Chapter 10).

Cost Lever	Applies To	Trade-off
Spot VMs + checkpointing	Training jobs	Cheaper, but the job can be preempted and must resume
Batch prediction instead of an endpoint	Latency-tolerant scoring	Cheaper, but not real-time
Endpoint min replicas = 0	Rarely used endpoints	Cheaper when idle, adds cold-start latency
Narrower hyperparameter search	Tuning jobs	Faster/cheaper, slightly less exhaustive search
Idle shutdown on notebooks	Vertex AI Workbench	No downside if genuinely idle

16.4 Budgets, alerts, and labels

Google Cloud **Budgets and alerts** let a team set a monthly spending threshold per project or per billing account and receive notifications (or, with a Cloud Function, automatically trigger a response, such as disabling billing) as actual spend approaches it. **Labels** — simple key-value tags attached to resources — let a team break down a shared project's bill by team, environment, or experiment, which is usually the first step toward finding out which specific training jobs or endpoints are actually driving cost.

Chapter Recap

- Apply least-privilege IAM, VPC Service Controls, and CMEK as the default security posture for ML workloads.
- Sensitive Data Protection (DLP) discovers and redacts PII before it reaches training.
- The main cost levers are Spot VMs, batch over online prediction where possible, right-sized endpoint minimums, narrower hyperparameter search, and notebook idle shutdown.
- Budgets/alerts and resource labels are the standard tools for keeping spend visible and attributable.

PART 6

Exam Preparation

Every chapter, re-sorted by exam domain, plus a full practice exam, glossary, and cheat sheet.

CHAPTER 17

Exam Domains Mapped to Vertex AI Services

This chapter re-sorts everything you have learned by exam domain rather than by project stage, so your final review follows the same structure as the exam itself. For each domain, we list the Vertex AI services most likely to appear and the chapter where each is covered in depth.

Domain 1 — Architecting low-code ML solutions

- AutoML (Tables, Image, Video, Text, Translation) — Chapter 8
- BigQuery ML — Chapter 8
- Choosing AutoML vs. custom training vs. BigQuery ML — Chapter 8

Domain 2 — Collaborating within and across teams

- Vertex AI Workbench (instances vs. managed notebooks) — Chapter 7
- Vertex AI Feature Store as a shared, consistent feature layer — Chapter 6
- Model Registry as a shared model catalogue — Chapter 12

Domain 3 — Scaling prototypes into ML models

- Custom training: pre-built vs. custom containers, worker pools, distributed training — Chapter 9
- Hyperparameter tuning and Vertex AI Vizier — Chapter 10
- Moving from notebook prototypes to submitted training jobs — Chapter 7

Domain 4 — Serving and scaling models

- Online prediction via Endpoints, autoscaling, traffic splitting — Chapter 13
- Batch prediction — Chapter 13
- Vector Search for similarity/embedding-based retrieval — Chapter 15

Domain 5 — Automating and orchestrating ML pipelines

- Vertex AI Pipelines, KFP SDK vs. TFX — Chapter 11
- Model Registry and CI/CD, canary/shadow rollout patterns — Chapter 12
- Scheduling and event-triggered retraining — Chapter 11

Domain 6 — Monitoring, optimizing, and maintaining ML solutions

- Model Monitoring: skew vs. drift — Chapter 14
- Explainable AI and Responsible AI — Chapter 14
- Security, cost management, and quotas — Chapter 16

EXAM TIP — Cross-cutting foundations

Chapters 1–6 (Foundations and Data) are not their own exam domain, but they are prerequisites referenced inside almost every domain's scenario questions — IAM and service accounts in particular show up inside Domain 2, 3, 4, and 5 questions, not just as a topic of their own.

Chapter Recap

- Use this chapter's mapping, alongside Chapter 1's domain table, to target review time in your final two weeks before the exam.
- IAM, service accounts, and data preparation are prerequisite knowledge tested inside almost every domain.

CHAPTER 18

Practice Exam — 40 Questions with Explanations

These questions mirror the style, length, and scenario-based phrasing of the real exam. Work through them without looking at the answer first, then read the explanation even for questions you got right — the reasoning is often more valuable than the answer itself. Questions are grouped loosely by domain, matching Chapter 17.

Q1. A retail company has a small data team with no deep ML expertise and a tabular dataset of past sales. They need a demand-forecasting model quickly and want Google to handle model selection and tuning. What should they use?

A. Vertex AI custom training with a hand-written TensorFlow model
B. AutoML Tables (Vertex AI Tabular AutoML)
C. Vertex AI Vizier standalone optimisation
D. A self-managed Kubeflow cluster on GKE

Correct answer: B — AutoML Tables is designed exactly for this: structured data, limited ML expertise, and a need for automatic architecture and hyperparameter search.

Q2. A team already stores its transaction data in BigQuery and wants the fastest path to a working classification model using only SQL. What should they use?

A. AutoML Tables
B. BigQuery ML
C. Vertex AI Workbench with scikit-learn
D. Vertex AI Pipelines

Correct answer: B — BigQuery ML trains models directly with SQL statements against data already in BigQuery, without moving the data or writing Python.

Q3. Which scenario is the strongest reason to choose custom training over AutoML?

A. The team wants a fast baseline model within a day
B. The dataset is small and tabular
C. The project needs a novel model architecture and full control over training internals
D. The team has no ML engineers

Correct answer: C — AutoML abstracts away architecture and training internals; when you need a novel architecture or full interpretability of the training process, custom training is appropriate.

Q4. A data science team needs a shared place to store engineered features so that both training and real-time serving use identical, point-in-time-correct values. What should they use?

A. Cloud Storage buckets with CSV exports
B. Vertex AI Feature Store
C. Vertex AI Vector Search
D. BigQuery views only

Correct answer: B — Feature Store is purpose-built to serve identical feature definitions and point-in-time-correct values to both training (offline) and serving (online) consumers.

Q5. A Vertex AI Workbench instance is left running continuously and driving up cost even though it is rarely used. What is the best fix?

A. Switch to a smaller machine type
B. Enable idle shutdown
C. Move to AutoML instead
D. Disable the aiplatform API

Correct answer: B — The waste here is idle time, not machine size; idle shutdown automatically stops the instance when it is not being used.

Q6. Which Vertex AI resource is best described as "the single source of truth for every trained model version and which endpoint it is deployed to"?

A. Vertex AI Feature Store
B. Vertex ML Metadata
C. Vertex AI Model Registry
D. Vertex AI Vizier

Correct answer: C — The Model Registry tracks model versions, their evaluation metrics, and their deployment status across endpoints.

Q7. A data scientist has a working model prototype in a Vertex AI Workbench notebook. What is the recommended next step to make training reproducible and scalable?

A. Keep running the notebook manually whenever a new model is needed
B. Submit the code as a Vertex AI custom training job (eventually as a pipeline step)
C. Copy the notebook to every team member's laptop
D. Convert the notebook directly into an AutoML dataset

Correct answer: B — Extracting the working code into a script or container and submitting it as a managed training job (and later a pipeline step) is the standard path from prototype to production.

Q8. A team needs to train a very large deep learning model that does not fit in the memory of a single machine, even with multiple GPUs. What should they configure?

A. A single worker pool with more GPUs
B. Multiple worker pools with a distribution strategy for distributed training
C. AutoML Tables with a larger training budget
D. Vertex AI Vizier with more trials

Correct answer: B — When a single machine cannot hold the model, distributed training across multiple worker pools, using a distribution strategy in the training code, is required.

Q9. Which hyperparameter search strategy does Vertex AI Vizier use by default, and why is it generally preferred for expensive training jobs?

A. Grid search, because it is exhaustive
B. Bayesian optimisation, because it uses prior trial results to choose better future trials
C. Random search, because it is the simplest to implement
D. Manual search, because it requires no infrastructure

Correct answer: B — Vizier's Bayesian optimisation builds a probabilistic model from prior trials to intelligently choose the next trial, converging faster than grid or random search when each trial is expensive.

Q10. A hyperparameter tuning job is taking too long and costing too much. What is the best first fix?

A. Add more trials to explore further
B. Narrow the search space and enable parallel trials/early stopping
C. Switch to grid search for thoroughness
D. Increase the machine size for every trial

Correct answer: B — Narrowing the search space, running trials in parallel, and enabling early stopping are the standard levers to make tuning faster and cheaper without simply adding more, larger trials.

Q11. A company needs to score 8 million customer records for churn risk once per week, with results needed by the next morning but not instantly. What is the most cost-effective approach?

A. Deploy the model to an always-on Vertex AI Endpoint and call it 8 million times
B. Use Vertex AI Batch Prediction against a BigQuery table
C. Use AutoML training every week
D. Use Vertex AI Vector Search

Correct answer: B — This is a latency-tolerant, bulk workload — the textbook case for batch prediction, which avoids the continuous cost of an always-on endpoint.

Q12. An online endpoint must respond to fraud-check requests in under 200 milliseconds for live transactions. Which serving approach fits?

A. Vertex AI Batch Prediction
B. A Vertex AI Endpoint with online prediction
C. A nightly BigQuery ML scoring job
D. Vertex AI Data Labeling

Correct answer: B — Real-time, low-latency, synchronous requests are exactly what Vertex AI Endpoints (online prediction) are designed for.

Q13. An endpoint experiences noticeable latency spikes on the first request after periods of no traffic. What configuration change addresses this?

A. Set the minimum replica count to 0
B. Set the minimum replica count to 1 or more
C. Switch to batch prediction
D. Disable autoscaling entirely

Correct answer: B — A minimum of zero replicas saves money when idle but causes a cold start; keeping at least one replica running avoids that latency spike at the cost of continuous billing.

Q14. Which Vertex AI service is purpose-built for approximate nearest-neighbour search over millions of embedding vectors, commonly used for semantic search and recommendations?

A. Vertex AI Feature Store
B. Vertex AI Vector Search
C. Vertex AI Model Monitoring
D. Vertex AI Vizier

Correct answer: B — Vector Search (formerly Matching Engine) performs fast approximate nearest-neighbour search over embeddings at scale.

Q15. A company wants every model retraining run to be automatically triggered when new data lands in a Cloud Storage bucket, and wants full lineage from raw data to deployed model. What should they build?

A. A cron job on a Compute Engine VM that reruns a notebook

B. A Vertex AI Pipeline triggered by an event, tracked in Vertex ML Metadata

C. A manual checklist for the data science team

D. An AutoML dataset refreshed by hand

Correct answer: B — Vertex AI Pipelines, triggered by a Cloud Storage event via Eventarc, provides both automated, event-driven retraining and automatic lineage tracking through Vertex ML Metadata.

Q16. What is the key structural difference between the Kubeflow Pipelines SDK and TFX on Vertex AI?

A. TFX cannot run on Vertex AI at all

B. KFP is a general-purpose pipeline SDK you build components for yourself; TFX offers pre-built, ML-specific components

C. KFP only supports image models; TFX only supports tabular models

D. There is no meaningful difference

Correct answer: B — KFP is flexible and general-purpose; TFX is more opinionated with a rich library of pre-built ML pipeline components (ExampleGen, Trainer, Evaluator, etc.).

Q17. A team wants to test a new model version against a small percentage of live traffic before fully replacing the current production model, while being able to quickly compare business metrics. What deployment pattern fits best?

A. Shadow deployment

B. Canary deployment with traffic splitting

C. Full replacement with no testing

D. Batch prediction only

Correct answer: B — Canary deployment routes a small, real slice of live traffic to the new version, letting the team observe real business impact before a full rollout.

Q18. A team wants to evaluate a new model version's predictions against real traffic without any chance of it affecting what users actually see. What pattern fits?

A. Canary deployment

B. Shadow deployment

C. Immediate full rollout

D. A/B testing with 50/50 traffic split

Correct answer: B — Shadow deployment logs and compares the new model's predictions against live traffic without ever serving them to users, eliminating any user-facing risk.

Q19. A model's live prediction traffic is compared against the original training dataset and found to have significantly different statistical distributions for several features. What is this called?

A. Prediction drift

B. Training-serving skew

C. Feature leakage

D. Class imbalance

Correct answer: B — Training-serving skew is specifically the divergence between the training data distribution and the live serving data distribution.

Q20. A model's live prediction traffic distribution is compared against its own traffic from a month ago and found to have shifted significantly. What is this called, and what should happen next?

A. Training-serving skew; retrain immediately without investigation

B. Prediction drift; investigate and likely trigger a retraining pipeline

C. Feature leakage; discard the model

D. Class imbalance; resample the training set

Correct answer: B — A distribution shift measured against the model's own past traffic (not training data) is drift, typically the trigger for an automated or manual retraining pipeline run.

Q21. A Vertex AI custom training job needs to read from a specific Cloud Storage bucket and write results to a specific BigQuery dataset. What is the best-practice way to grant access?

A. Grant the Owner role on the project to the default Compute Engine service account

B. Create a dedicated service account with only the specific roles needed on those resources, and attach it to the job

C. Share your own personal credentials with the training job

D. Make the bucket and dataset publicly readable and writable

Correct answer: B — Least privilege: a dedicated service account scoped to exactly the resources and roles required, not a broad basic role at the project level.

Q22. Which Google Cloud concept lets a company enforce “no publicly readable storage buckets” across every project in an organization at once?

A. IAM custom roles

B. Organization Policy

C. Vertex AI Model Registry

D. Cloud Build triggers

Correct answer: B — Organization Policy is applied at the organization or folder level and is inherited by every project underneath, enforcing rules like disallowing public buckets.

Q23. A dataset has a strong time dimension (transactions over the past two years). What splitting strategy avoids leaking future information into the training set?

A. A purely random 80/10/10 split

B. A chronological split, with the most recent data held out for testing

C. Splitting by customer ID alphabetically

D. No split — use all data for training

Correct answer: B — Random splits can leak future information into training when data is time-ordered; a chronological split (train on older data, test on newer data) mirrors how the model will actually be used.

Q24. A classification dataset is 99% negative class and 1% positive class. Which evaluation metric is most misleading to rely on alone?

A. Precision
B. Recall
C. Raw accuracy
D. Area under the precision-recall curve

Correct answer: C — A model predicting the majority class every time would score 99% accuracy while being useless — precision, recall, F1, or PR-AUC are more informative for imbalanced classes.

Q25. A training pipeline's evaluation accuracy is suspiciously perfect, far better than any reasonable model should achieve, and drops sharply once deployed. What is the most likely cause?

A. Underfitting
B. Feature/data leakage — information from after the prediction moment leaked into training features
C. Class imbalance
D. Insufficient hyperparameter tuning

Correct answer: B — Suspiciously high training/evaluation performance that does not hold up in production is a classic sign of leakage, often solved with point-in-time-correct feature retrieval (Feature Store).

Q26. Which Vertex AI feature specifically helps prevent feature leakage when assembling a training set from historical labels and engineered features?

A. Vertex AI Vizier
B. Feature Store's point-in-time lookup in the offline store
C. Vertex AI Explainable AI
D. Vertex AI Model Monitoring

Correct answer: B — Point-in-time correct batch reads from the offline Feature Store ensure feature values reflect what was true at each label's historical timestamp, not the present.

Q27. A model needs to explain, for a specific loan application, how much each input feature contributed to the decision. Which Vertex AI capability provides this?

A. Vertex AI Vizier
B. Vertex Explainable AI (feature attributions, e.g. Shapley values)
C. Vertex AI Vector Search
D. Vertex AI Data Labeling

Correct answer: B — Explainable AI attributes a share of a specific prediction's outcome to each input feature, using Shapley values for tabular models.

Q28. Which technique produces a pixel-level saliency map explaining an image classification model's prediction?

A. Shapley values
B. Integrated Gradients / XRAI
C. Bayesian optimisation
D. Point-in-time lookups

Correct answer: B — Integrated Gradients and XRAI are the Explainable AI techniques used for image models to highlight which pixels most influenced a prediction.

Q29. A healthcare company needs to remove or mask personally identifiable information from a dataset before training. Which service should they use?

A. Vertex AI Vizier
B. Sensitive Data Protection (Cloud DLP)
C. Vertex AI Vector Search
D. Vertex AI Feature Store

Correct answer: B — Sensitive Data Protection (formerly Cloud DLP) discovers and can redact, mask, or tokenise PII before it reaches a training pipeline.

Q30. Which cost-saving technique is most appropriate for a large, non-urgent training job that checkpoints regularly?

A. On-demand standard VMs only
B. Spot VMs (preemptible)
C. A larger, more expensive machine type
D. Disabling checkpointing to save storage cost

Correct answer: B — Spot VMs cost significantly less and are appropriate for non-urgent, checkpointed training jobs that can tolerate and resume from preemption.

Q31. A company wants to keep all prediction traffic to a Vertex AI Endpoint off the public internet for compliance reasons. What should they use?

A. A public HTTPS endpoint with an API key
B. Private Service Connect
C. Vertex AI Data Labeling
D. AutoML Tables

Correct answer: B — Private Service Connect allows prediction traffic to stay entirely within a VPC, never traversing the public internet.

Q32. What is the primary purpose of resource labels (key-value tags) on Google Cloud resources in an ML project?

A. They encrypt the resource
B. They enable cost attribution and breakdown by team, environment, or experiment
C. They are required for IAM to function
D. They automatically scale endpoints

Correct answer: B — Labels let a shared project's bill be broken down and attributed to specific teams, environments, or experiments, which is usually the first step in finding what is driving cost.

Q33. A team is unsure whether to use grid search, random search, or Bayesian optimisation for a very expensive-to-train model with five hyperparameters. Which is generally most efficient?

A. Grid search, because it is exhaustive
B. Random search, because it requires no configuration
C. Bayesian optimisation (Vertex AI Vizier's default), because it uses prior results to guide future trials
D. Manual trial and error

Correct answer: C — For expensive training, Bayesian optimisation typically converges on good hyperparameters with fewer trials than grid or random search.

Q34. What does the AIP_MODEL_DIR environment variable provide inside a Vertex AI custom training job?

A. The IAM role of the job's service account

B. A Cloud Storage path the training script should write the final model artifacts to

C. The region the job is running in

D. The machine type of the job

Correct answer: B — Vertex AI automatically sets AIP_MODEL_DIR (and similar variables) so training code can write artifacts to a durable location without hardcoding paths.

Q35. Which statement correctly distinguishes Vertex AI Feature Store from Vertex AI Vector Search?

A. They are the same service under two names

B. Feature Store performs exact key lookups of named feature values; Vector Search performs approximate nearest-neighbour similarity search over embeddings

C. Feature Store is for images only; Vector Search is for tabular data only

D. Vector Search replaced Feature Store and Feature Store is deprecated

Correct answer: B — Feature Store answers “what is this named feature's value for this entity” (exact lookup); Vector Search answers “which items are most similar to this one” (approximate nearest neighbour over embeddings).

Q36. A pipeline step trains a model, and a later step should only deploy it if its accuracy exceeds the currently deployed version's accuracy. What Vertex AI concept enables this conditional promotion pattern?

A. AutoML dataset splitting

B. Comparing evaluation metrics recorded in the Model Registry as part of the pipeline's conditional logic

C. Vertex AI Vizier trials

D. Cloud DLP redaction

Correct answer: B — The Model Registry stores each version's evaluation metrics, which a pipeline's conditional step can compare against the currently deployed version before deciding whether to redeploy.

Q37. Which of the following is the best description of Vertex ML Metadata's role in a pipeline?

A. It stores IAM policies for the project

B. It automatically records lineage — which data, code, and parameters produced each artifact and model

C. It hosts the online Feature Store

D. It performs hyperparameter tuning

Correct answer: B — Vertex ML Metadata automatically tracks artifacts and their relationships across pipeline runs, giving full lineage from raw data to a deployed model.

Q38. A company's ML team wants every code change to training logic to be automatically tested before it can affect the production model. What should they implement?

A. Manual code review only, with no automated testing
B. A CI/CD system (e.g. Cloud Build) that runs tests and gates changes before triggering a pipeline run
C. Direct edits to the production training script with no review
D. Disabling the Model Registry to simplify deployment

Correct answer: B — CI/CD for ML automatically tests changes to training code and pipeline definitions before they can reach production, complementing (not replacing) pipeline automation.

Q39. A model trained primarily on one demographic performs noticeably worse for another demographic despite good overall accuracy. What practice would have caught this earlier?

A. Relying solely on aggregate accuracy
B. Evaluating performance sliced by relevant subgroups, alongside fairness metrics
C. Increasing the learning rate
D. Switching to AutoML

Correct answer: B — Responsible AI practice requires evaluating performance sliced by subgroup, not only in aggregate, to catch exactly this kind of disparity.

Q40. Which choice best completes: "Use ___ for real-time, low-latency predictions and ___ for large-scale, latency-tolerant, scheduled scoring."

A. Batch prediction; online endpoints
B. Online endpoints (Vertex AI Endpoints); Batch prediction
C. AutoML; custom training
D. Feature Store; Vector Search

Correct answer: B — This is the core online-vs-batch distinction from Chapter 13: endpoints for real-time, low-latency serving; batch prediction for large-scale, scheduled, latency-tolerant scoring.

Chapter Recap

- 40 scenario-style questions covering all six exam domains — revisit any you missed by chapter number.
- If you scored below 80%, re-read the corresponding chapters before your exam date rather than moving straight to more practice questions.

Glossary

A quick-reference glossary of terms used throughout this course, in alphabetical order.

AutoML
Vertex AI's automated model-building service; searches architectures, features, and hyperparameters automatically given labeled data and a target column.

Artifact
A tracked input or output of a pipeline step or training job — a dataset, model, or metric — recorded in Vertex ML Metadata.

Batch prediction
Asynchronous, large-scale scoring of a dataset with no persistent endpoint; cheaper and slower than online prediction.

BigQuery ML
Trains a range of model types using SQL statements directly against BigQuery data.

Canary deployment
Routing a small percentage of live traffic to a new model version to validate it before a full rollout.

CI/CD for ML
Automated testing and controlled promotion of changes to training code, pipelines, and models before they affect production.

CMEK
Customer-Managed Encryption Keys; lets an organisation control the encryption key used for data at rest instead of relying solely on Google-managed keys.

Custom training
Running your own training code (script or container) on managed Vertex AI compute, as opposed to AutoML's automated approach.

Data leakage / feature leakage
Accidentally including information in training features that would not be available at prediction time, inflating evaluation metrics unrealistically.

Endpoint
A persistent, managed service hosting one or more deployed model versions for online, synchronous predictions.

Entity type / Feature (Feature Store)
An entity type is the “thing” features describe (e.g. user); a feature is a single named attribute of that entity type.

Explainable AI
Vertex AI's capability for attributing a prediction to individual input features (Shapley values for tabular, Integrated Gradients/XRAI for images).

Feature Store
A centralized, versioned repository of engineered features, serving both training (offline) and real-time serving (online) consistently.

Hyperparameter
A training configuration value (e.g. learning rate) set before training and not learned from data.

IAM
Identity and Access Management; controls which principal (user, group, service account) can do what to which Google Cloud resource.

Kubeflow Pipelines (KFP) SDK
A general-purpose Python SDK for authoring pipeline DAGs, compiled to run on Vertex AI Pipelines.

Model Monitoring
Vertex AI service that detects training-serving skew and prediction drift by comparing live traffic statistics against training data or past traffic.

Model Registry
The catalogue of every trained model version, its metrics, and its deployment status across endpoints.

Organization Policy
A constraint set at the organization or folder level, inherited by every project beneath it.

Point-in-time correctness
Retrieving feature values exactly as they were at a specific historical timestamp, preventing feature leakage.

Prediction drift
Statistical divergence between current live traffic and the model's own past live traffic, measured continuously.

Preemptible / Spot VM
A discounted virtual machine that Google can reclaim with little notice; well-suited to non-urgent, checkpointed training.

Service account
A non-human identity used by software (a training job, pipeline, or endpoint) to authenticate and act under IAM permissions.

Shadow deployment
Running a new model version alongside the current one on live traffic, comparing predictions offline, without ever serving the new version's output to users.

TFX (TensorFlow Extended)
An opinionated, ML-specific pipeline framework with pre-built components (ExampleGen, Trainer, Evaluator, etc.), compiled to run on Vertex AI Pipelines.

Training-serving skew
Statistical divergence between the training data distribution and the live serving data distribution, measured once against the original training set.

Vertex AI Pipelines
A managed service for executing DAGs of containerised, versioned, reproducible ML workflow steps.

Vertex AI Vector Search (Matching Engine)
A managed service for fast, approximate nearest-neighbour search over large collections of embedding vectors.

Vertex AI Vizier
Google's black-box optimisation service, powering Vertex AI's built-in hyperparameter tuning via Bayesian optimisation.

Vertex AI Workbench
Vertex AI's managed JupyterLab notebook environment, pre-integrated with BigQuery, Cloud Storage, Feature Store, and training.

VPC Service Controls
A security perimeter that restricts data movement out of a defined set of Google Cloud resources, even for otherwise-authorised identities.

Appendix — One-Page Cheat Sheet

A condensed, exam-morning review of the highest-yield distinctions from this course. If a term here is unfamiliar, return to its chapter before your exam.

If the scenario says…	Think…
Limited ML expertise, standard tabular/image/text problem	AutoML
Team already fluent in SQL, data lives in BigQuery	BigQuery ML
Novel architecture, full control needed	Custom training
Notebook cost is out of control	Idle shutdown
Model needs multiple machines to train	Distributed training, multiple worker pools
Expensive training, need efficient tuning	Vertex AI Vizier (Bayesian optimisation)
Real-time, low-latency predictions	Vertex AI Endpoint (online prediction)
Large-scale, scheduled, latency-tolerant scoring	Batch prediction
Cold-start latency on a rarely used endpoint	Set minimum replicas ≥ 1
Similar items / semantic search / RAG retrieval	Vertex AI Vector Search
Consistent feature values for training and serving	Vertex AI Feature Store
Suspiciously perfect training accuracy	Suspect feature/data leakage
Automate retraining on new data or a schedule	Vertex AI Pipelines
Track every model version and its metrics	Vertex AI Model Registry
Test a new model on a slice of real users	Canary deployment
Test a new model with zero user impact	Shadow deployment
Compare live traffic to original training data	Training-serving skew
Compare live traffic to itself over time	Prediction drift
Explain why a model made a specific prediction	Explainable AI (Shapley / Integrated Gradients)
Non-urgent training, checkpoints regularly	Spot VMs
PII must be removed before training	Sensitive Data Protection (DLP)
Keep prediction traffic off the public internet	Private Service Connect
Least-privilege access for a training job	Dedicated service account, scoped IAM roles

Final reminder: the exam rewards judgement about which managed service fits a described situation, more than memorised trivia. When two answers both seem technically possible, prefer the one that best

matches the specific constraints mentioned in the scenario — cost, latency, team expertise, data type, or scale — exactly as this course has emphasised chapter by chapter. Good luck.